Selecting an embedding model should be based on benchmark results obtained from the target application's own dataset rather than adopting a model simply because it performs well elsewhere.

Design principle 2: Thresholds and safety checks

Unlike traditional caching, semantic caching does not operate as a simple match-or-no-match system. Instead, responses are retrieved based on similarity scores, making the choice of distance threshold a critical design

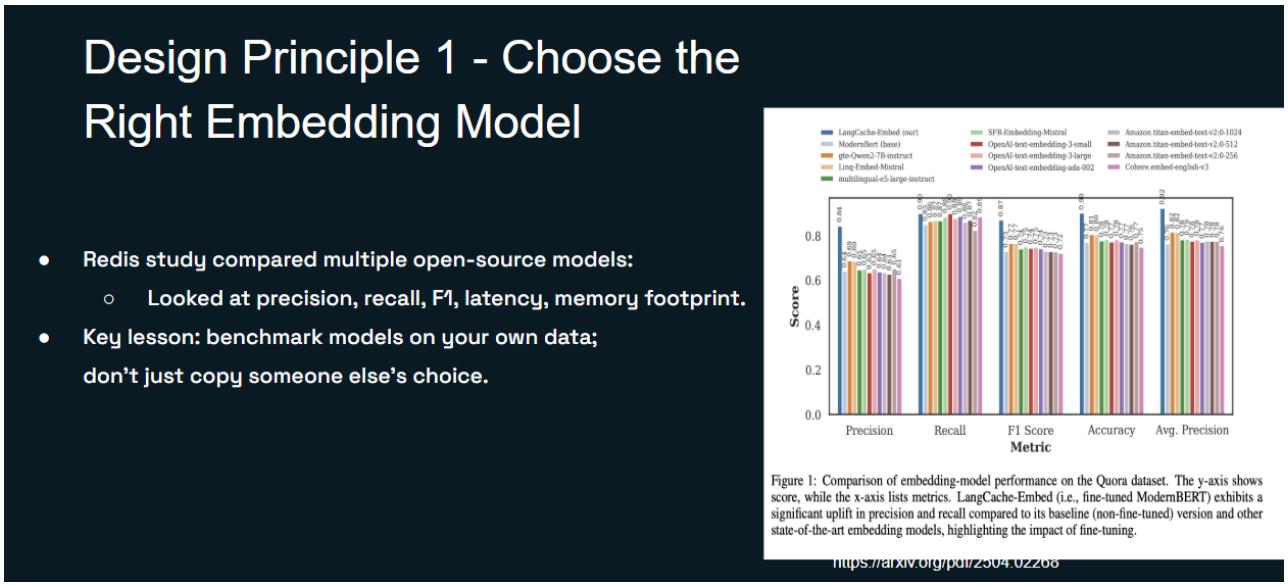

Figure 1: Choosing the right embedding model is critical for semantic caching

When semantic caching works best

Semantic caching delivers the greatest benefits when users repeatedly ask similar questions.

Applications such as customer support systems, chatbots and virtual assistants often receive recurring requests with different wording but identical intent. In these environments, semantic caching can significantly improve response times while lowering inference costs.

It is particularly effective when:

- User queries frequently overlap in meaning.
- Low latency is important for user experience.
- Domain knowledge remains relatively stable over time.
- Policy documents, FAQs, or support information change infrequently.
- A vector database is already part of the application architecture or can be integrated easily.

As more requests are cached, the cache hit ratio increases, allowing a greater proportion of responses to be served without repeatedly invoking an LLM.

When semantic caching is not suitable

Despite its advantages, semantic caching is not appropriate for every application.

Rapidly changing information, such as live sports scores, financial market prices or continuously updated sensor readings, is generally unsuitable because cached responses become outdated quickly.

Similarly, highly personalised information should not be stored in a semantic cache. Queries such as checking an individual's bank balance or retrieving private account information require user-specific responses and cannot be safely reused across different users.

Applications that receive mostly unique requests with very little repetition are also unlikely to benefit significantly from semantic caching, since cache hit rates remain low.

Selecting the appropriate use cases is essential to achieving meaningful improvements in performance and cost.